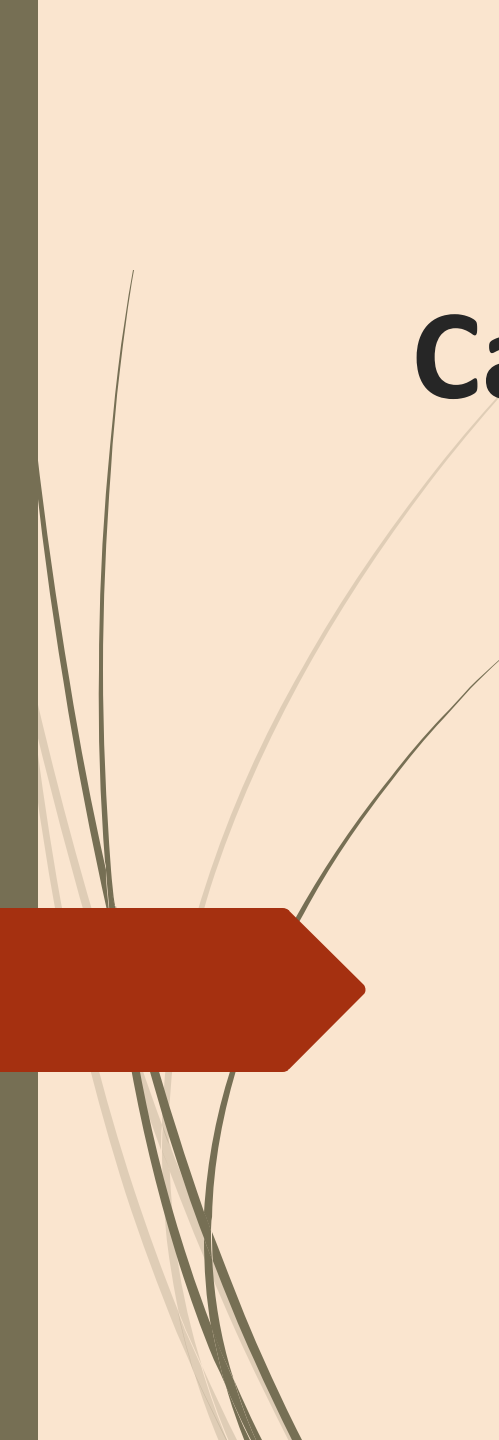




Otaku Verse

Catalogue manga, anime et jeux vidéo

Node • HTML/CSS • React



Architecture technique

Backend

- ▀ Node.js
- ▀ Express
- ▀ Middleware de validation et d'upload
- ▀ API Rest

Base de données

- ▀ MySQL
- ▀ Sequelize

Frontend

- ▀ HTML / CSS / Javascript
- ▀ React / React Router / Vite



Fonctionnalités développées

Accueil

En tant que visiteur, je veux :

- Affichage des contenus récents
- Accès aux pages détails
- Recherche simple et rapide

Catalogue

En tant qu'utilisateur, je veux :

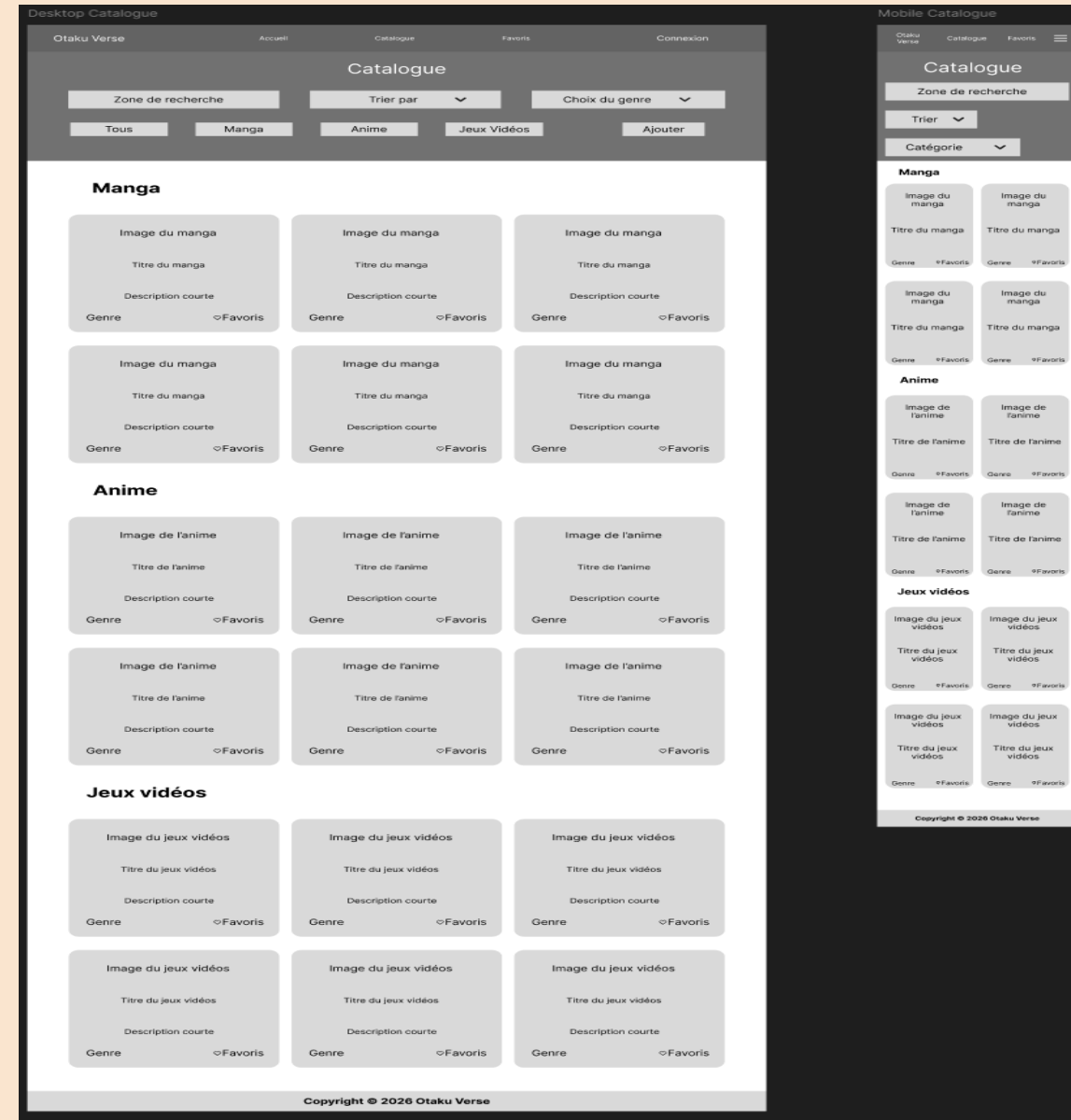
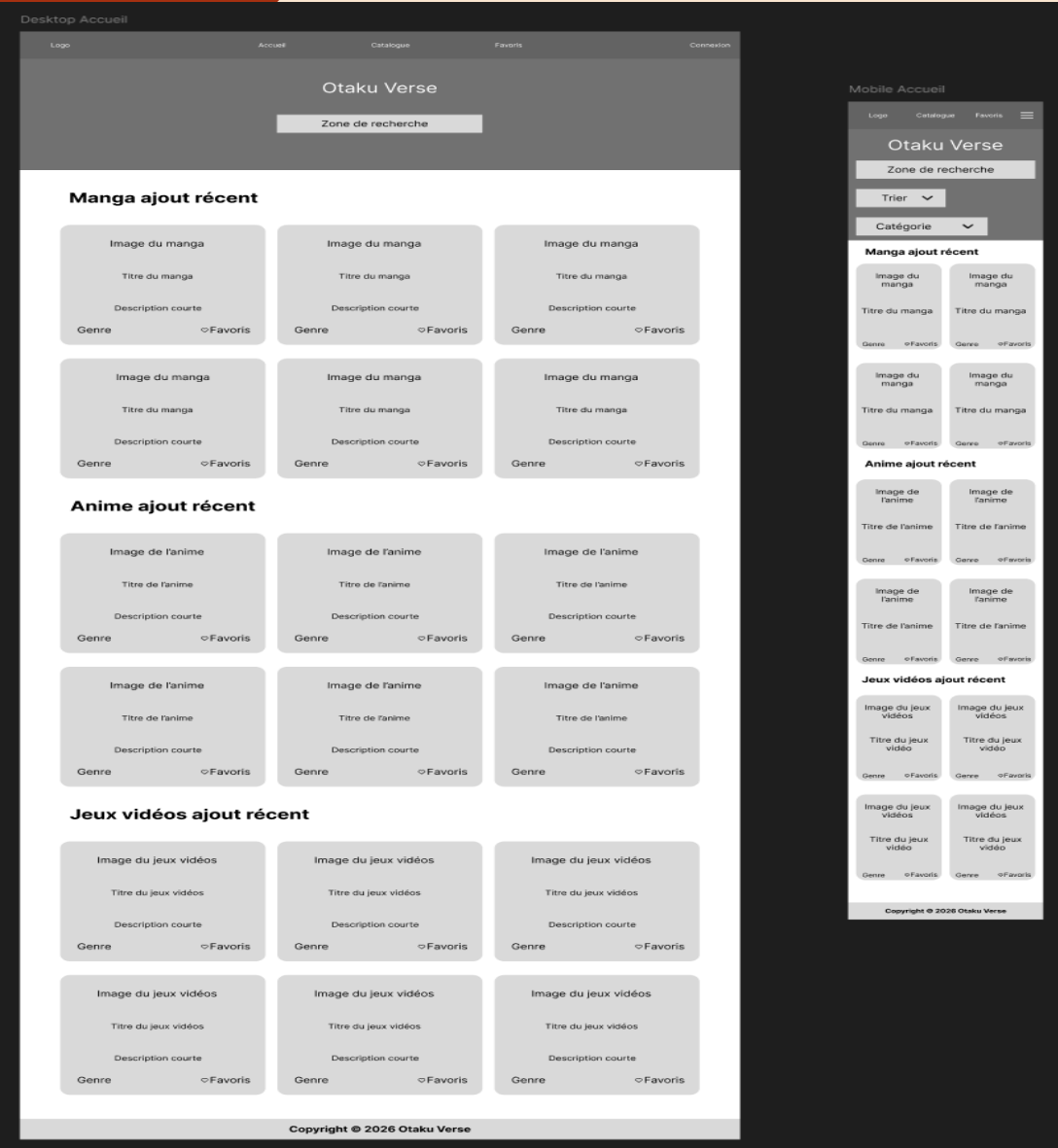
- Consultation de tout le contenu du site
- Accès aux pages détails
- Filtrage par catégorie, genre et fonction de tri

Gestion du contenu

En tant qu'admin, je veux :

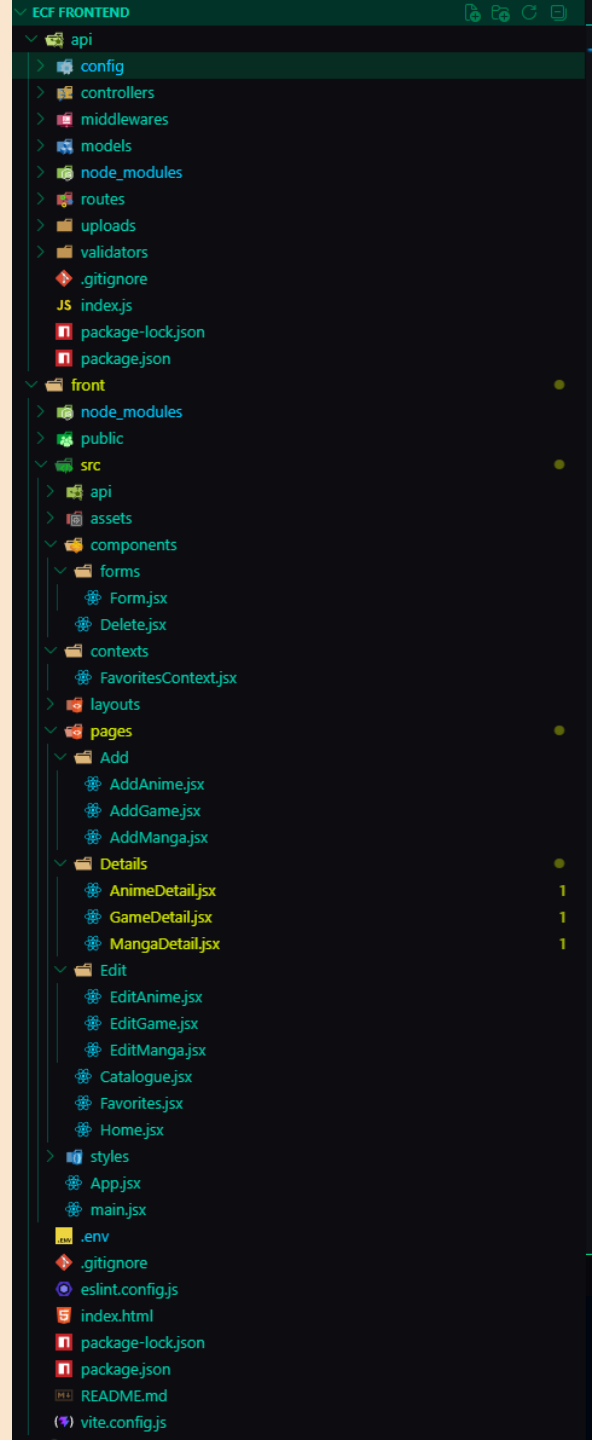
- Ajout de contenu
- Modification et suppression du contenu
- Gestion des favoris

Figma



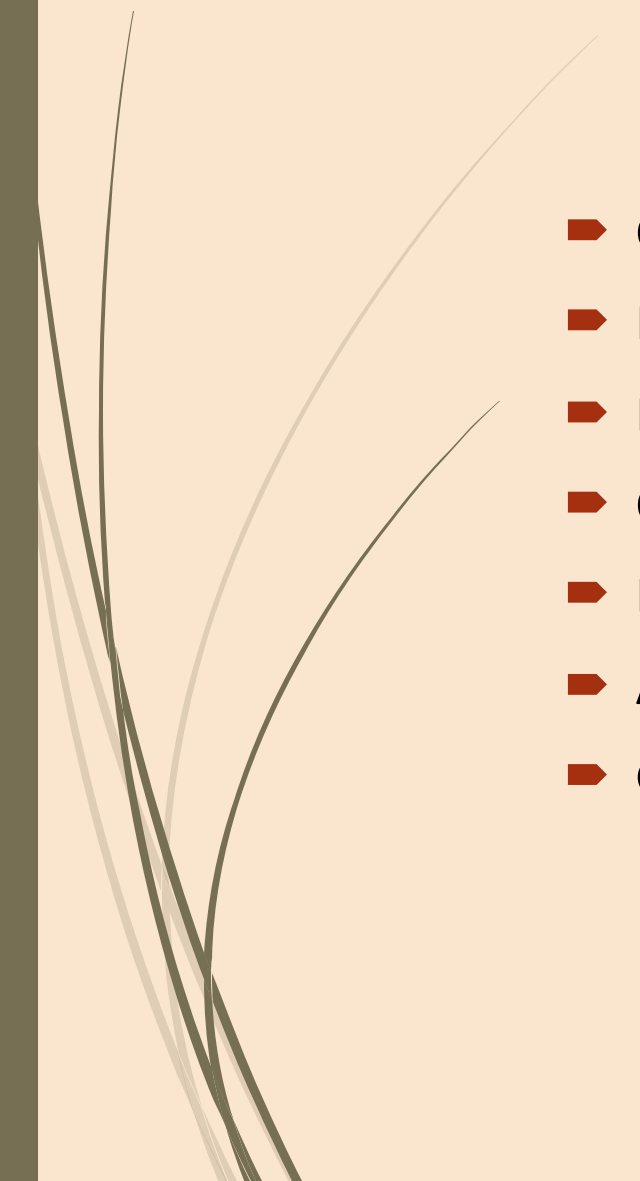
Mise en œuvre initiale du projet

- Mise en place de l'environnement React / Vite
- Création du backend avec Node.js et Express
- Connexion à la BDD MySQL
- Développement du CRUD
- Création Accueil + Catalogue
- Mis en place des pages détails
- Mis en place des composants Form et Delete
- Ajout des filtres de tri + gestion des favoris





Mise en place du backend avec API

- Création du serveur backend avec Node.js et Express
 - Mise en place des routes API
 - Développement des contrôleurs pour mangas, animes et jeux vidéo
 - Connexion à la base de données MySQL
 - Mise en place des modèles de données
 - Ajout de la validation des données
 - Gestion de l'upload des images
- 

Mise en place de la page Accueil

- Création de la page d'accueil en React
- Mise en place d'une barre de recherche
- Affichage des mangas les plus récents
- Affichage des animes les plus récents
- Affichage des jeux vidéo les plus récents
- Accès rapide aux pages détail et modification

```
1 // You'll need a 2nd server to run this
2 import { useState, useEffect } from "react";
3 import { Link } from "react-router";
4 import { fetchGames, deleteGame } from "../api/game";
5 import { fetchMangas, deleteManga } from "../api/manga";
6 import { fetchAnimes, deleteAnime } from "../api/anime";
7 import { useFavorite } from "../contexts/favoritesContext";
8
9 const API_URL = import.meta.env.VITE_API_URL;
10
11 Windurf Refactor | Explain | Generate JSDoc | X
12 const Home = () => {
13   const [games, setGames] = useState([]);
14   const [search, setSearch] = useState("");
15   const [mangas, setMangas] = useState([]);
16   const [animes, setAnimes] = useState([]);
17   const [isFavorite, toggleFavorite] = useFavorite();
18
19   Windurf Refactor | Explain | Generate JSDoc | X
20   const removeGame = async (id) => {
21     await deleteGame(id);
22   }
23
24   const data = await fetchGames(search);
25   setGames(data);
26
27   Windurf Refactor | Explain | Generate JSDoc | X
28   const removeManga = async (id) => {
29     await deleteManga(id);
30   }
31
32   const data = await fetchMangas(search);
33   setMangas(data);
34
35   Windurf Refactor | Explain | Generate JSDoc | X
36   const removeAnime = async (id) => {
37     await deleteAnime(id);
38   }
39
40   const data = await fetchAnimes(search);
41   setAnimes(data);
42
43   useEffect(() => {
44     fetchGames(search).then(data => setGames(data));
45   }, [search]);
46
47   useEffect(() => {
48     fetchMangas(search).then(data => setMangas(data));
49   }, [search]);
50
51   useEffect(() => {
52     fetchAnimes(search).then(data => setAnimes(data));
53   }, [search]);
54 }
```

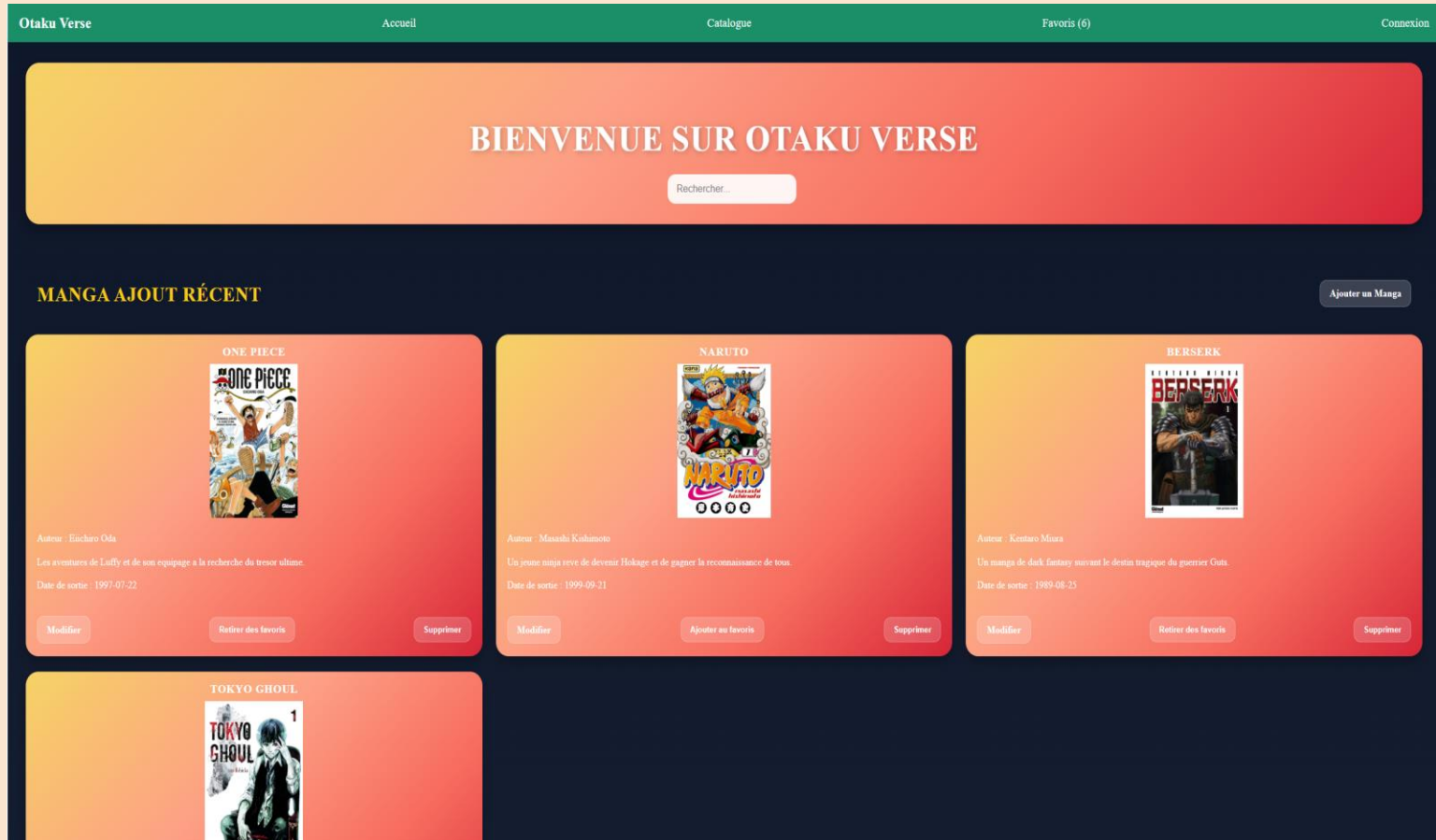
```
return (
  <main>
    <header className="hero">
      <h1 className="title">Bienvenue sur Otaku Verse</h1>
      <form className="search-form" role="search">
        <label htmlFor="search">
          <input id="search" type="search" placeholder="Rechercher..." onChange={(e) => setSearch(e.target.value)} />
        </label>
      </form>
    </header>
    <div>
      <div>
        <h2 id="manga-heading">Manga ajout récent</h2>
        <Link to="/add/manga">Ajouter un Manga</Link>
      </div>
      <ul className="media-list">
        {Array.isArray(mangas) && mangas
          .sort((a, b) => new Date(b.createdAt) - new Date(a.createdAt))
          .slice(0, 6)
          .map((manga) => (
            <li key={manga.id}>
              <article className="media-card">
                <h3 className="media-card_title">
                  <Link to={`/manga/${manga.id}`} className="media-card_link">{manga.title}</Link>
                </h3>
                <figure className="media-card_image">
                  <img src={`/${API_URL}/${manga.image}`} alt={manga.title} />
                </figure>
                <div className="media-card_content">
                  <p className="media-card_author">Auteur : {manga.author}</p>
                  <p className="media-card_description">{manga.description}</p>
                  <p className="media-card_date">Date de sortie : {manga.release_year}</p>
                </div>
                <div className="media-card_actions">
                  <Link to={`/edit/manga/${manga.id}`}>Modifier</Link>
                  <button onClick={() => toggleFavorite({ ...manga, type: "manga" })}>
                    {isFavorite(manga.id, "manga") ? "Retirer des favoris" : "Ajouter au favoris"}
                  </button>
                  <button onClick={() => removeManga(manga.id)}>Supprimer</button>
                </div>
              </article>
            </li>
          ))}
      </ul>
    </div>
  </main>
)
```

```

    Section Animes */)
<section className="media-section" aria-labelledby="anime-heading">
  <div className="media-card_actions">
    <h2 id="anime-heading">Animes ajout récent</h2>
    <Link to="/add/anime">Ajouter un Anime</Link>
  </div>
  <ul className="media-list">
    {Array.isArray(animes) && animes
      .sort((a, b) => new Date(b.createdAt) - new Date(a.createdAt))
      .slice(0, 6)
      .map((anime) => (
        <li key={anime.id}>
          <article className="media-card">
            <h3 className="media-card_title">
              <Link to={` /anime/${anime.id}`} className="media-card_link">{anime.title}</Link>
            </h3>
            <figure className="media-card_image">
              <img src={` ${API_URL}/${anime.image}`} alt={anime.title} />
            </figure>
            <div className="media-card_content">
              <p className="media-card_description">{anime.description}</p>
            </div>
            <footer className="media-card_actions">
              <Link to={` /edit/anime/${anime.id}`}>Modifier</Link>
              <button onClick={() => toggleFavorite({ ...anime, type: "anime" })}>
                {isFavorite(anime.id, "anime") ? "Retirer des favoris" : "Ajouter au favoris"}
              </button>
              <button onClick={() => removeAnime(anime.id)}>Supprimer</button>
            </footer>
          </article>
        </li>
      ))}
  </ul>
</section>

/* Section Jeux vidéos */
<section className="media-section" aria-labelledby="games-heading">
  <div className="media-card_actions">
    <h2 id="games-heading">Jeux vidéos ajout récent</h2>
    <Link to="/add/game">Ajouter un Jeux Vidéo</Link>
  </div>
  <ul className="media-list">
    {Array.isArray/games) && games
      .sort((a, b) => new Date(b.createdAt) - new Date(a.createdAt))
      .slice(0, 6)
      .map((game) => (
        <li key={game.id}>
          <article className="media-card">
            <h3 className="media-card_title">
              <Link to={` /game/${game.id}`} className="media-card_link">{game.title}</Link>
            </h3>
            <figure className="media-card_image">
              <img src={` ${API_URL}/${game.image}`} alt={game.title} />
            </figure>
            <div className="media-card_content">
              <p className="media-card_description">{game.description}</p>
            </div>
            <footer className="media-card_actions">
              <Link to={` /edit/game/${game.id}`}>Modifier</Link>
              <button onClick={() => toggleFavorite({ ...game, type: "game" })}>
                {isFavorite(game.id, "game") ? "Retirer des favoris" : "Ajouter au favoris"}
              </button>
              <button onClick={() => removeGame(game.id)}>Supprimer</button>
            </footer>
          </article>
        </li>
      ))}
  </ul>
</section>

```





Création du Catalogue

- Affichage des mangas, animes et jeux vidéo
- Mise en place d'une recherche par mot-clé
- Ajout de filtres par genre, catégorie et trier par
- Tri et affichage des contenus sous forme de cartes
- Accès aux pages détail
- Accès à la modification et suppression des contenus

CATALOGUE

Rechercher...

GenreTous

Trier parTitre A → Z

- Tous
- Anime
- Manga
- Jeux videos

MANGA

Ajouter un Manga

BERSERK



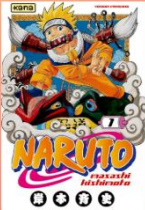
Un manga de dark fantasy suivant le destin tragique du guerrier Guts.

Modifier

Retirer des favoris

Supprimer

NARUTO



Un jeune ninja rêve de devenir Hokage et de gagner la reconnaissance de tous.

Modifier

Ajouter au favoris

Supprimer

ONE PIECE



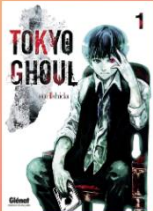
Les aventures de Luffy et de son équipage à la recherche du trésor ultime.

Modifier

Retirer des favoris

Supprimer

TOKYO GHOUL



Un étudiant devient un humain mi-ouïe après une transplantation qui change sa vie.

Gestion des favoris

- Ajout aux favoris
- Retrait des favoris
- Mise à jour dynamique du bouton
- Intégration sur les pages détail

MES FAVORIS

MANGAS FAVORIS

ONE PIECE



Auteur : Eiichiro Oda

Les aventures de Luffy et de son équipage à la recherche du trésor ultime.

Date de sortie : 1997-07-22

Retirer des favoris

BERSERK



Auteur : Kentaro Miura

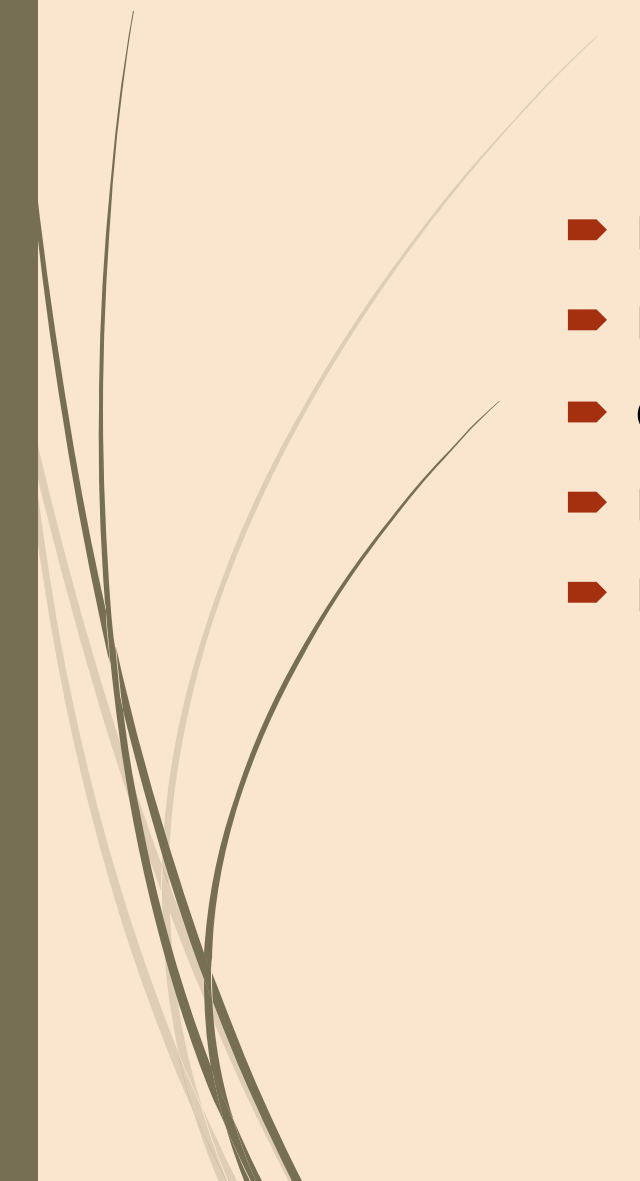
Un manga de dark fantasy suivant le destin tragique du guerrier Guts.

Date de sortie : 1989-08-25

Retirer des favoris



Mise en place des Formulaire d'ajout et de modification

- Formulaires dédiés pour mangas, animes et jeux vidéo
 - Récupération dynamique des catégories et genres
 - Gestion des champs texte, date et image
 - Envoi des données au backend
 - Redirection après ajout ou modification
- 

Ajouter un manga

Ajouter un anime

Retour à l'accueil

TITRE

DATE DE SORTIE

jj/mm/aaaa

AUTEUR

DESCRIPTION

NOMBRE DE VOLUMES

CATÉGORIE

— Choisir une catégorie —

GENRE

— Choisir un genre —

IMAGE

Choisir un fichier

Aucun fichier choisi

Ajouter un manga

TITRE

DATE DE SORTIE

jj/mm/aaaa

AUTEUR

DESCRIPTION

NOMBRE D'ÉPISODES

CATÉGORIE

— Choisir une catégorie —

GENRE

— Choisir un genre —

IMAGE

Choisir un fichier

Aucun fichier choisi

Ajouter un anime

Ajouter un jeux vidéo

Retour à l'accueil

TITRE

DATE DE SORTIE

jj/mm/aaaa

AUTEUR

DESCRIPTION

CATÉGORIE

— Choisir une catégorie —

GENRE

— Choisir un genre —

IMAGE

Choisir un fichier

Aucun fichier choisi

Ajouter un jeu vidéo

MODIFIER LE MANGA

Retour à l'accueil

TITRE

One Piece

DATE DE SORTIE

22/07/1997

AUTEUR

Eiichiro Oda

DESCRIPTION

Les aventures de Luffy et de son équipage à la recherche du trésor ultime.

NOMBRE DE VOLUMES

108

CATÉGORIE

— Choisir une catégorie —

GENRE

— Choisir un genre —

IMAGE

Choisir un fichier

Aucun fichier choisi

MODIFIER LE MANGA

MODIFIER L'ANIME

Retour à l'accueil

TITRE

Attack on Titan

DATE DE SORTIE

07/04/2013

AUTEUR

Hajime Isayama

DESCRIPTION

Un anime sombre où l'humanité lutte pour survivre face à des titans géants.

NOMBRE D'ÉPISODES

89

CATÉGORIE

— Choisir une catégorie —

GENRE

— Choisir un genre —

IMAGE

Choisir un fichier

Aucun fichier choisi

MODIFIER L'ANIME

MODIFIER LE JEUX VIDÉO

Retour à l'accueil

TITRE

Battlefield 6

DATE DE SORTIE

19/10/2025

AUTEUR

DICE

DESCRIPTION

Battlefield 6 est un jeu vidéo de tir à la première personne développé par DICE, Criterion, Motive et Ripple Effect et édité par Electronic Arts

CATÉGORIE

— Choisir une catégorie —

GENRE

— Choisir un genre —

IMAGE

Choisir un fichier

Aucun fichier choisi

MODIFIER LE JEUX VIDÉO



Améliorations

- Mettre en place la gestion utilisateur
 - Ajouter une authentification sécurisée
 - Créer un espace administrateur
 - Ajouter une pagination au catalogue
 - Améliorer la version mobile du site
 - Renvoyer vers des sites de lecture en ligne, streaming et achat
 - Ajouter plus d'options de tri et de filtres
 - Améliorer les retours visuels en cas d'erreur
- 